

```
# Experiment 3 Data Analytics I
#Descriptive Statistics -Measures of CentralTendency and variability
#Perform thefollowing operations onany open source dataset(e.g.,data.csv)
#1. Provide summary statistics (mean, median, minimum, maximum, standard deviation) for a dataset (age, income etc.)
#with numeric variables grouped by one of the qualitative (categorical) variable. For example, if your categorical v
#2. Write a Python program to display some basic statistical details like percentile, mean, standard deviation etc.
#of the species of 'Iris-setosa', 'Iris-versicolor' and 'Iris- versicolor' of iris.csv dataset.
#Provide the codes with outputs and explain everything thatyou doing this step.
```

```
#1. Provide summary statistics (mean, median, minimum, maximum, standard deviation) for a dataset (age, income etc.)
```

```
import numpy
```

```
age = [99,86,87,88,111,86,103,87,94,78,77,85,86]
```

```
x = numpy.mean(age)
```

```
print(x)
```

```
89.76923076923077
```

```
import numpy
```

```
age = [99,86,87,88,111,86,103,87,94,78,77,85,86]
```

```
x = numpy.median(age)
```

```
print(x)
```

```
87.0
```

```
import numpy
```

```
age = [99,86,87,88,86,103,87,94,78,77,85,86]
```

```
x = numpy.median(age)
```

```
print(x)
```

```
86.5
```

```
from scipy import stats
```

```
age = [99,86,87,88,111,86,103,87,94,78,77,85,86]
```

```
x = stats.mode(age)
```

```
print(x)
```

```
ModeResult(mode=np.int64(86), count=np.int64(3))
```

```
n_num = [1, 2, 3, 4, 5]
```

```
n = len(n_num)
```

```
get_sum = sum(n_num)
```

```
mean = get_sum/n
```

```
print("Mean / Average is : " + str(mean))
```

```
Mean / Average is : 3.0
```

```
n_num = [1, 2, 3, 4, 5]
```

```
n = len(n_num)
```

```
n_num.sort()
```

```
if n % 2 == 0:
```

```
    median1 = n_num[n//2]
```

```
    median2 = n_num[n//2-1]
```

```
    median = (median1 + median2)/2
```

```
else:
```

```
median = n_num[n//2]
print("Median is : " + str(median))
```

Median is : 3

```
from collections import Counter

n_num = [1, 2, 3, 4, 5, 5]
n = len(n_num)

data = Counter(n_num)

get_mode = dict(data)
mode = [k for k, v in get_mode.items() if v == max(list(data.values()))]

if len(mode) == n:
    get_mode = "No mode found"
else:
    get_mode = "Mode is/are : " + ', '.join(map(str, mode))
print(get_mode)
```

Mode is/are : 5

```
import pandas as pd

df = pd.DataFrame({'A' : ['a', 'b', 'c', 'c', 'a', 'b'],
                   'B' : [0, 1, 1, 0, 1, 0]}, dtype = "category")

df.dtypes
```

0

A category

B category

dtype: object

```
print(df)

print(df.groupby(['A']). count().reset_index())
```

```
  A  B
0  a  0
1  b  1
2  c  1
3  c  0
4  a  1
5  b  0
  A  B
0  a  2
1  b  2
2  c  2
```

/tmp/ipykernel_1770/1566825704.py:3: FutureWarning: The default of observed=False is deprecated and will be changed to observed=True in a future version of pandas. Consider specifying the value of observed explicitly.

```
print(df.groupby(['A']). count().reset_index())
```

```
import pandas as pd

df = pd.DataFrame({'A' : ['a', 'b', 'c', 'c', 'a', 'b'],
                   'B' : [0, 1, 1, 0, 1, 0],
                   'C' : [7, 8, 9, 5, 3, 6]})

df['A'] = df['A'].astype('category')

print(df)

print(df.groupby(['A', 'B']).mean().reset_index())
```

```
  A  B  C
0  a  0  7
1  b  1  8
2  c  1  9
3  c  0  5
4  a  1  3
5  b  0  6
  A  B  C
0  a  0  7.0
1  a  1  3.0
```

```
2 b 0 6.0
3 b 1 8.0
4 c 0 5.0
5 c 1 9.0
```

```
/tmp/ipykernel_1770/1222694041.py:11: FutureWarning: The default of observed=False is deprecated and will be changed
print(df.groupby(['A', 'B']).mean().reset_index())
```

#2. Write a Python program to display some basic statistical details like percentile, mean, standard deviation etc. #of the species of 'Iris-setosa', 'Iris-versicolor' and 'Iris- versicolor' of iris.csv dataset.
#Provide the codes with outputsandexplaineverything thatyou doin this step.

```
import pandas as pd

data = pd.read_csv(r"Iris.csv") # Corrected path from r"content/Iris.csv" to r"Iris.csv"

print('Iris-setosa')
setosa= data['Species'] == 'Iris-setosa'
print(data[setosa].describe())

print('\nIris-versicolor')
versicolor= data['Species'] == 'Iris-versicolor'
print(data[versicolor].describe())

print('\nIris-virginica')
virginica = data['Species'] == 'Iris-virginica'
print (data[virginica].describe())
```

```
Iris-setosa
count    Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
mean    50.00000    5.00600    3.418000    1.464000    0.24400
std     14.57738    0.35249    0.381024    0.173511    0.10721
min      1.00000    4.30000    2.300000    1.000000    0.10000
25%     13.25000    4.80000    3.125000    1.400000    0.20000
50%     25.50000    5.00000    3.400000    1.500000    0.20000
75%     37.75000    5.20000    3.675000    1.575000    0.30000
max     50.00000    5.80000    4.400000    1.900000    0.60000
```

```
Iris-versicolor
count    Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
mean    50.00000    5.936000    2.770000    4.260000    1.326000
std     14.57738    0.516171    0.313798    0.469911    0.197753
min     51.00000    4.900000    2.000000    3.000000    1.000000
25%     63.25000    5.600000    2.525000    4.000000    1.200000
50%     75.50000    5.900000    2.800000    4.350000    1.300000
75%     87.75000    6.300000    3.000000    4.600000    1.500000
max    100.00000    7.000000    3.400000    5.100000    1.800000
```

```
Iris-virginica
count    Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
mean    50.00000    6.58800    2.974000    5.552000    2.02600
std     14.57738    0.63588    0.322497    0.551895    0.27465
min    101.00000    4.90000    2.200000    4.500000    1.40000
25%    113.25000    6.22500    2.800000    5.100000    1.80000
50%    125.50000    6.50000    3.000000    5.550000    2.00000
75%    137.75000    6.90000    3.175000    5.875000    2.30000
max    150.00000    7.90000    3.800000    6.900000    2.50000
```

```
data.describe()
```

```
      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
count  150.000000      150.000000      150.000000      150.000000      150.000000
mean    75.500000         5.843333         3.054000         3.758667         1.198667
std     43.445368         0.828066         0.433594         1.764420         0.763161
min      1.000000         4.300000         2.000000         1.000000         0.100000
25%     38.250000         5.100000         2.800000         1.600000         0.300000
50%     75.500000         5.800000         3.000000         3.350000         1.300000
75%    112.750000         6.400000         3.300000         5.100000         1.800000
max    150.000000         7.900000         3.800000         6.900000         2.500000
```

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Id              150 non-null   int64
 1   SepalLengthCm   150 non-null   float64
 2   SepalWidthCm    150 non-null   float64
 3   PetalLengthCm   150 non-null   float64
 4   PetalWidthCm    150 non-null   float64
 5   Species         150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

```
from google.colab import files
```

```
uploaded = files.upload()
```

```
if uploaded:
    print("File uploaded successfully!")
else:
    print("No file uploaded.")
```

Iris.csv

Iris.csv(text/csv) - 5107 bytes, last modified: 3/29/2026 - 100% done

Saving Iris.csv to Iris.csv

File uploaded successfully!